

1. Given the quicksort algorithm using the Hoare and Lomuto partitions as presented in the lectures.

1. Provide one example for which Lomuto partition is faster than Hoare partition.

For array $A = [5, 5, 5, 5, 5]$ with index from 0 to 4.

- For Hoare partition: We need 3 swap actions
 - When $i = 0, j = 3$, we swap $A[i]$ with $A[j]$. Then $A = [5, 5, 5, 5, 5]$ with $i = 1, j = 2$.
 - When $i = 1, j = 2$, we swap $A[i]$ with $A[j]$. Then $A = [5, 5, 5, 5, 5]$ with $i = 2, j = 1$.
 - The partition finished with a swap $A[r]$ with $A[i]$
- For Lomuto partition: We need only 1 swap action. (But the partition is bad)
 - When $i = -1, j = 0$. Since $A[j] \leq A[r]$, then we don't swap.
 - When $i = -1, j = 1$. Since $A[j] \leq A[r]$, then we don't swap.
 - When $i = -1, j = 2$. Since $A[j] \leq A[r]$, then we don't swap.
 - When $i = -1, j = 3$. Since $A[j] \leq A[r]$, then we don't swap.
 - Since $j = 4 > r - 1$, we finished with a swap $A[i+1]$ with $A[r]$.

Thus Lomuto partition is faster since it performs less swap

2. Provide one example for which Hoare partition is faster than Lomuto partition.

For array $A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$ with index from 0 to 9.

- For Hoare partition: We only need 1 swap action.
 - When $i = 0, j = 8$. Since $A[i] < A[r]$ ($1 < 10$), we do $i++$ repeatedly (1, 2, ..., 9) until $i = 9$. Since $i > j$ ($9 > 8$), the loop finished
 - The partition finished with a swap $A[r]$ with $A[i]$ (swapping $A[9]$ with $A[9]$).
- For Lomuto partition: We need 10 swap actions.
 - When $i = -1, j = 0$. Since $A[j] < A[r]$ ($1 < 10$), then we do $i++$, then $i = 0$ and swap $A[j]$ with $A[i]$. $A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$
 - When $i = 0, j = 1$. Since $A[j] < A[r]$ ($2 < 10$), then we do $i++$, then $i = 1$ and swap $A[j]$ with $A[i]$. $A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$
 - When $i = 1, j = 2$. Since $A[j] < A[r]$ ($3 < 10$), then we do $i++$, then $i = 2$ and swap $A[j]$ with $A[i]$. $A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$
 - When $i = 2, j = 3$. Since $A[j] < A[r]$ ($4 < 10$), then we do $i++$, then $i = 3$ and swap $A[j]$ with $A[i]$. $A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$
 - When $i = 3, j = 4$. Since $A[j] < A[r]$ ($5 < 10$), then we do $i++$, then $i = 4$ and swap $A[j]$ with $A[i]$. $A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$
 - When $i = 4, j = 5$. Since $A[j] < A[r]$ ($6 < 10$), then we do $i++$, then $i = 5$ and swap $A[j]$ with $A[i]$. $A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$
 - When $i = 5, j = 6$. Since $A[j] < A[r]$ ($7 < 10$), then we do $i++$, then $i = 6$ and swap $A[j]$ with $A[i]$. $A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$
 - When $i = 6, j = 7$. Since $A[j] < A[r]$ ($8 < 10$), then we do $i++$, then $i = 7$ and swap $A[j]$ with $A[i]$. $A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$
 - When $i = 7, j = 8$. Since $A[j] < A[r]$ ($9 < 10$), then we do $i++$, then $i = 8$ and swap $A[j]$ with $A[i]$. $A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$
 - Since $j = 9 > r - 1$, we finished with a swap $A[i+1]$ with $A[r]$ (swapping $A[9]$ with $A[9]$).

Thus Hoare partition is faster since it performs less swap

3. Provide a general argument on when one partition approach is better than the other.

Hoare partition is better when the array has many duplicate values because when there are many duplicates, Hoare will swap and move index again and again, then we have a balanced partition (cut an array half and half). Thus the complexity is $O(\log n)$. However, even if Lomuto perform fewer swap for many duplicate values. It will result in the bad partition (partition is not half half) since Lomuto partition will not swap and move the index when there are duplicate values. In the worst case, each partition will split a size k array into 1 size array and $k - 1$ size array. Then the complexity is $O(n^2)$.

Thus, in general, Hoare partition is better than Lomuto partition since Hoare's complexity is better than Lomuto

Lomuto partition is better when you want a simple and understandable partition algorithm.

2. Provide the pseudo code of quicksort using the Hoare partition.

java

```
1 quickSort(A, l, r)
2   if l < r then
3       p = HoarePartition(A,l,r)
4       quickSort(A,l,p-1)
5       quickSort(A,p+1,r)
6   end if
7
8 HoarePartition(A,l,r)
9   i = l
10  j = r-1
11  pivot = A[r]
12  while i <= j do
13      while i <= j and A[i] < pivot do
14          i = i + 1
15      end while
16      while i <= j and A[j] > pivot do
17          j = j - 1
18      end while
19      if i <= j then
20          swap A[i] with A[j]
21          i = i + 1
22          j = j - 1
23      end if
24  end while
25  swap A[i] with A[r]
26 }
```

3. Implement quicksort using the Hoare partition and Lomuto partition in Java (the use of do while, while true, break and continue statements is strictly forbidden).

Finished

4. Provide a program that effectively shows that the runtime of the examples provided in item (a) are really faster when running the implementation. Hint: you may need to execute the sorting many times to collect average times.

Note!!! In 1(a),1(b) we give an array with small size. But in the test, we need to extend the size of array to avoid the noise.

In 1(a), our array is `A=[5,5,5,5,5]`, in the test we use `A'=[5,5,5,...,5]` with size 1000. By induction, we can also know lomuto partition is also faster than Hoare parittion for `A'` by argument in 1(a) with same idea

In 1(b), our array is `A=[1,2,3,4,5,6,7,8,9,10]`, in the test we use `A'=[0,1,2,...,999]` with size 1000. By induction, we can also know lomuto partition is also faster than Hoare parittion for `A'` by argument in 1(b) with same idea

Although in (a.1) array, Lomuto partition is faster than Hoare Partition. But since Lomuto partition is not balanced, then the sorting will cost more time as the test result shows: to sort (a,1) array, quicksort using lomuto partition is slower than quicksort using hoare partition although for the speed of partition itself, Lomuto is quicker.

For (a.2) array, quciksort using Lomuto partition is slower than quicksort using Hoare partition since Hoare Parition swap less than Lomuto as we analyzed in a. Also the partition for array are both generally balanced. Thus the result also verify this.